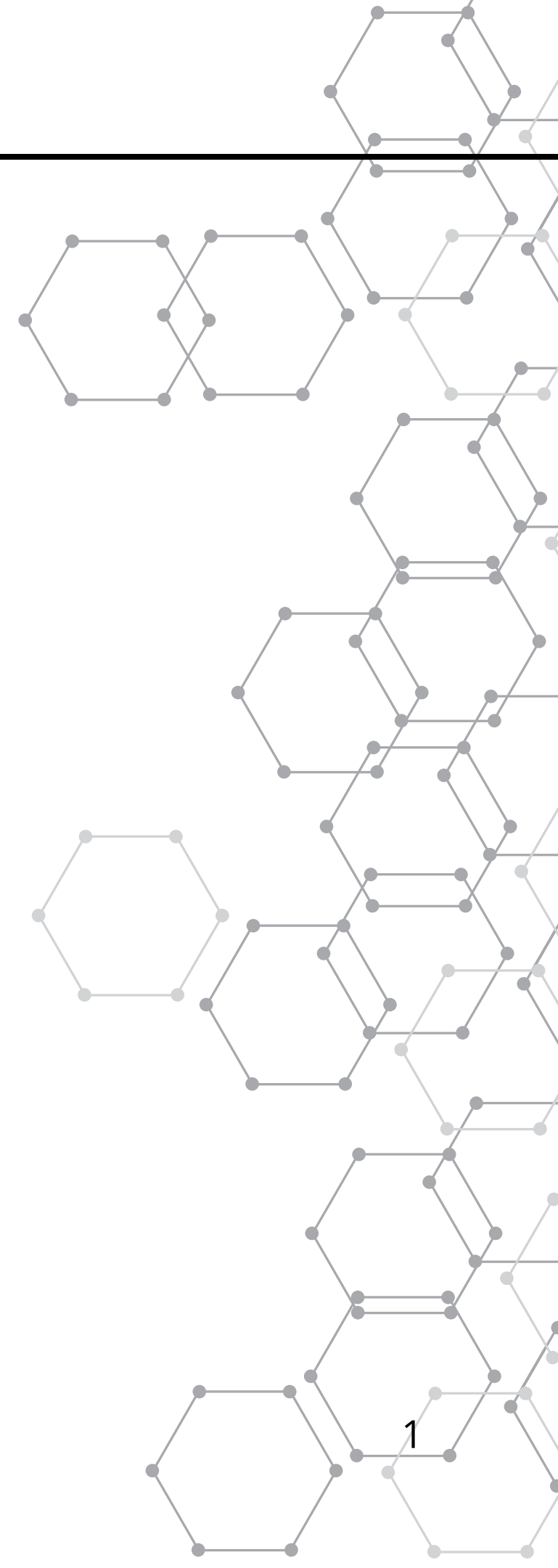
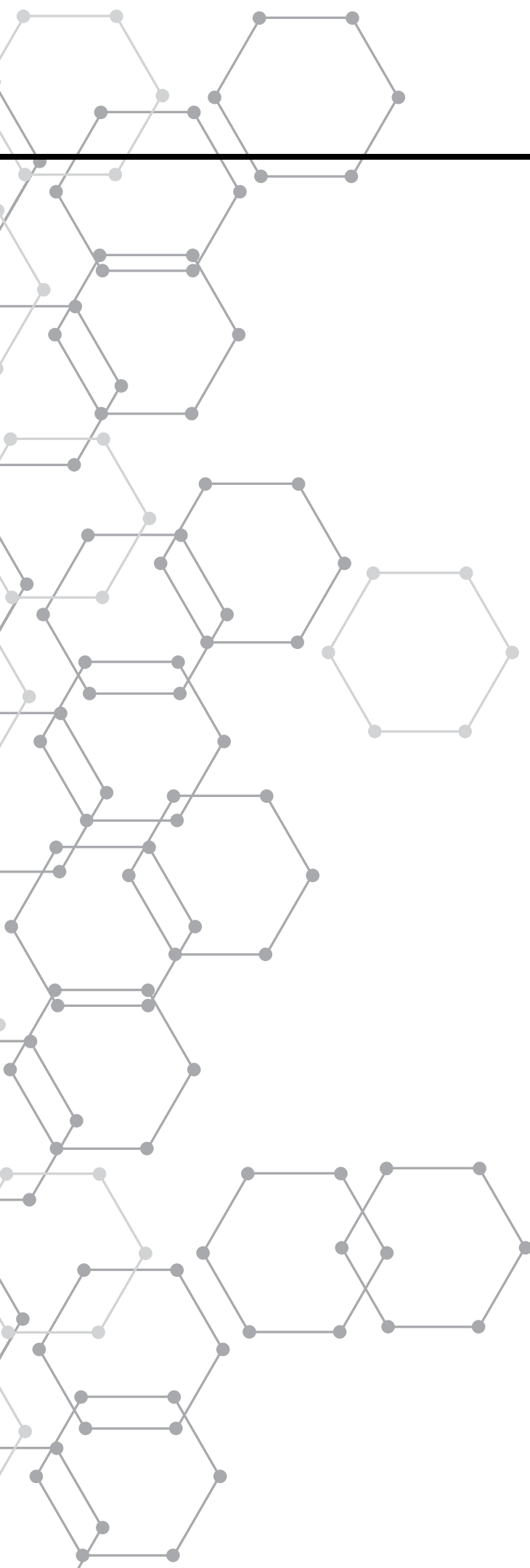




ROS

Robot Operating System (ROS) is an open-source ecosystem that provides the framework, tools, and libraries for building, deploying, running, and maintaining robotic applications.

This project provides an Architectural Analysis of ROS from the perspective of operating systems principles.

Irene González Quirós

1. System structure

- ROS2 as middleware simulating a microkernel
- Ross Client Library (analogy to Virtual File System)

2. Inter-Process Communication (IPC) through DDS

- Differences between classical IPC and DDS
- Topics in ROS

3. Two-level scheduling.

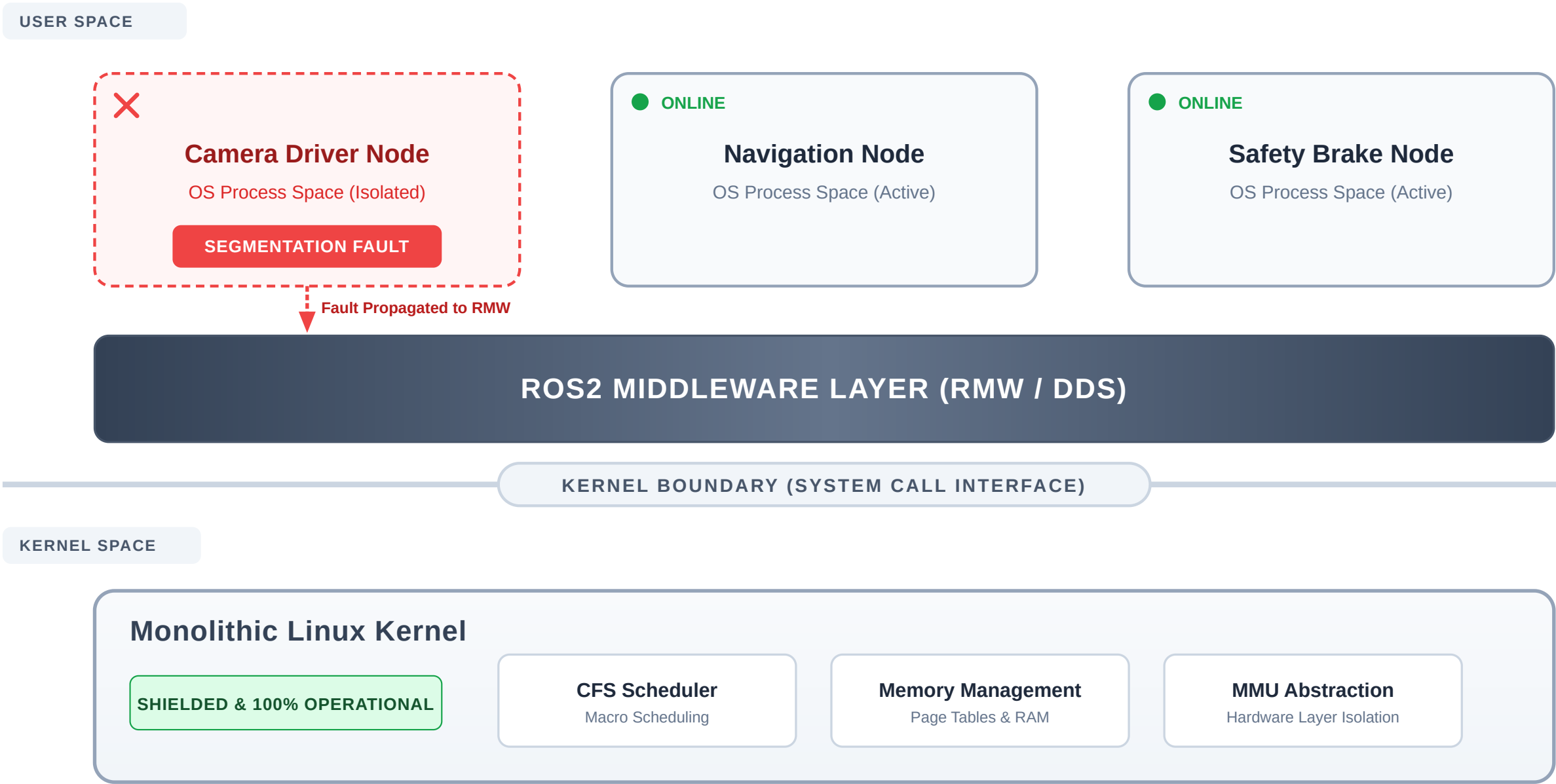
- Why ROS needs second level scheduling (executors)

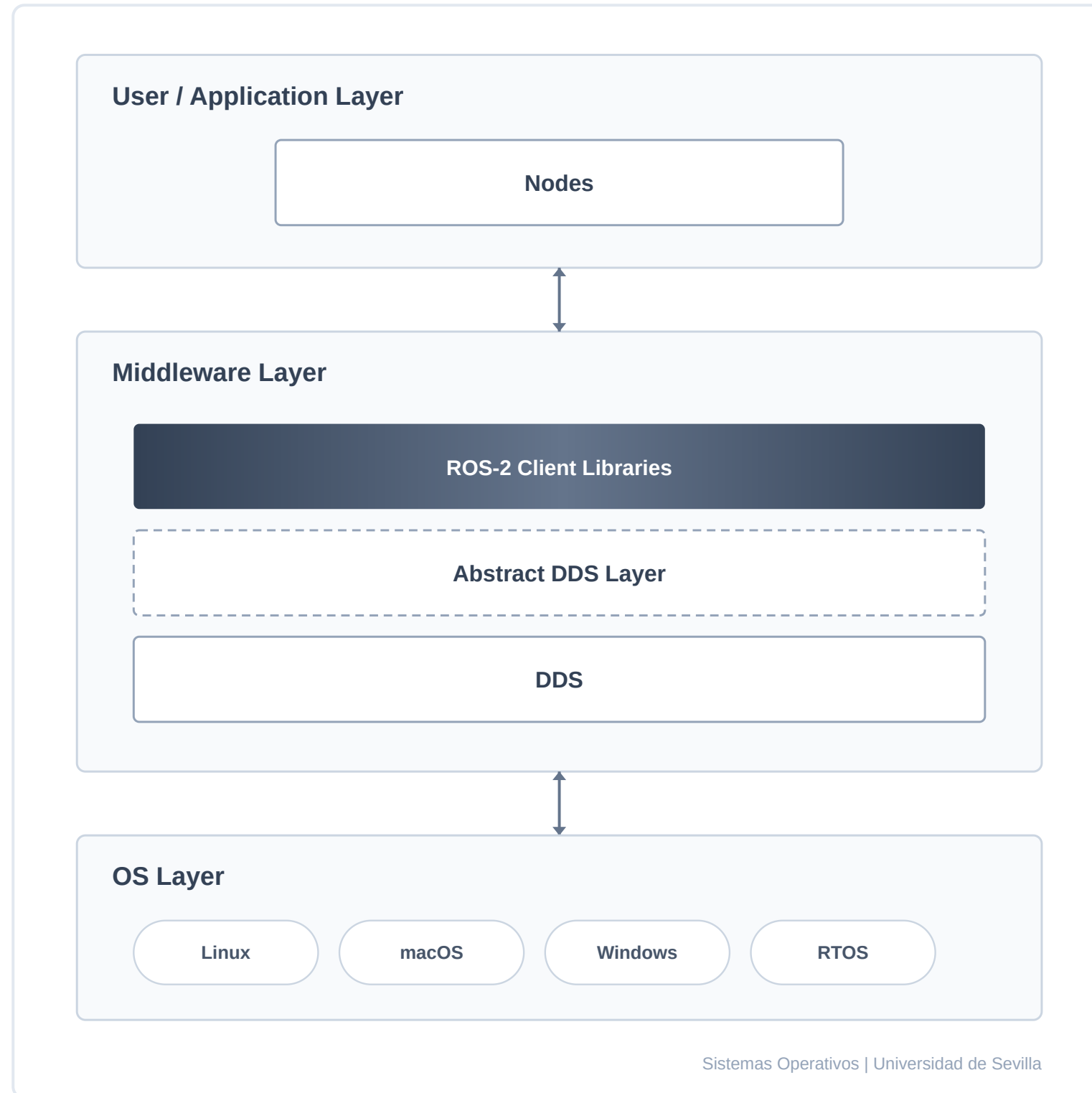
System structure

A standard monolithic kernel like Linux provides excellent execution efficiency, but doesn't allow for complex new functionalities or isolated maintenance.

Therefore, ROS2 is designed as a Middleware. Everything is executed in User Mode.

If a specific driver fails, it triggers a standard isolated software exception, shielding the underlying OS kernel from crashing.





RCL to VFS analogy

In a traditional OS, the Virtual File System (VFS) acts as a uniform interface layer.

A user program calls `read()` or `write()`, which works with any type of drive: EXT4, FAT32...

In ROS2, the ROS Client Library (RCL) does the exact same thing for robotics.

A developer calls `publish()` or `subscribe()`, which works on any type of hardware device.

It acts conceptually as a "Virtual Robot Layer" just like VFS acts as a Virtual File System.

Inter-Process Communication through DDS

Why pipes aren't enough

Classical OS pipes are limited to 1-to-1 communication. ROS2 requires broader distribution

Shared Memory implementation

For local nodes, DDS creates a shared memory segment, the way that traditional OS does.

Network sockets

For remote nodes, the middleware transparently transitions to network sockets (UDP/IP).

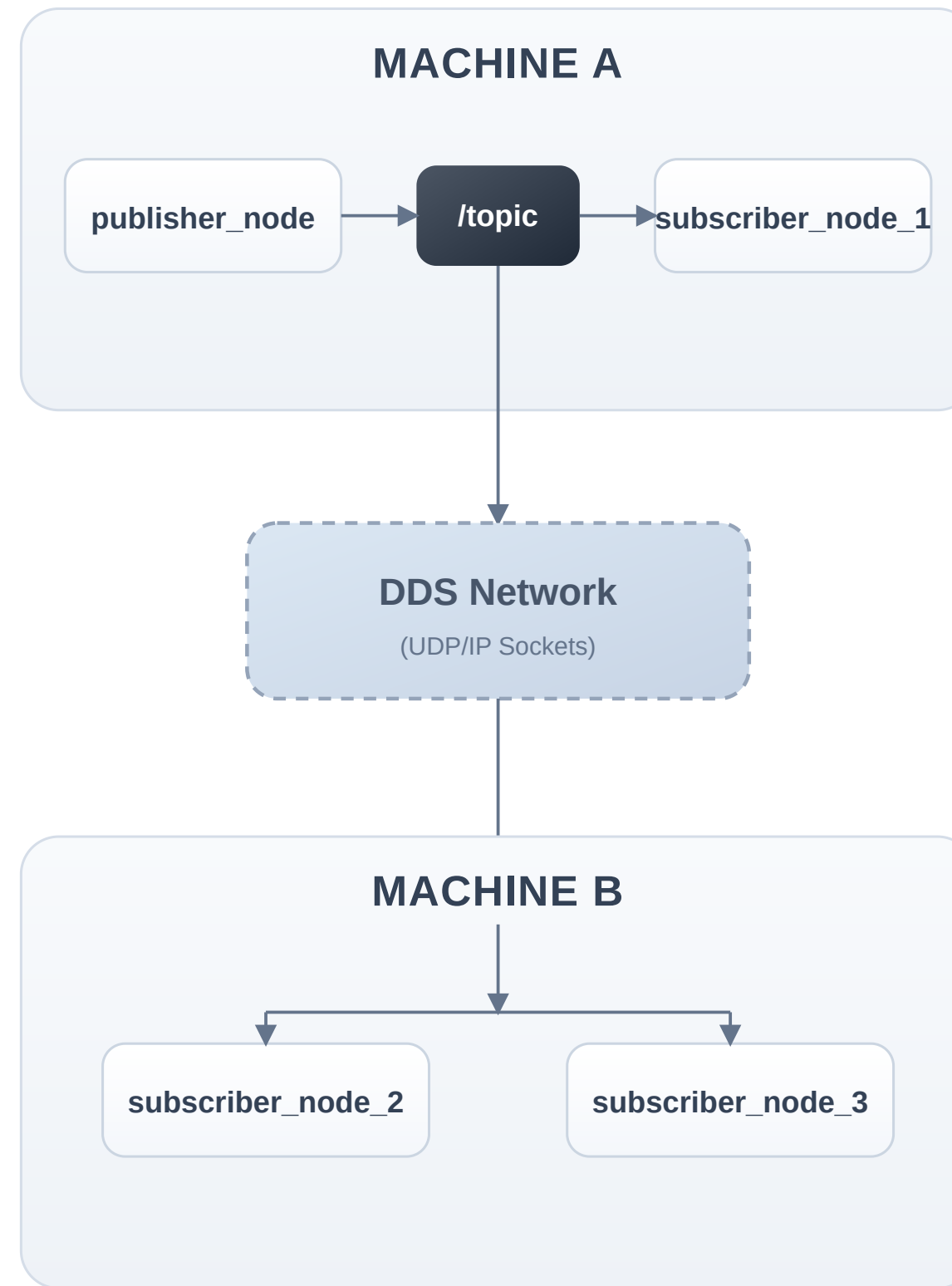
Data Distribution Service allows for expansions in traditional IPCs. It avoids the use of pipes, whilst maintaining shared memory implementation.

DDS Topics

The communication provided by DDS is possible due to the Interoperability Real-Time Publish-Subscribe Protocol (DDSI-RTPS).

This protocol follows a simple publish/subscribe paradigm. The idea is to establish secure and efficient communication between nodes, even in heterogeneous platforms.

The communication receives the name of “topics”.



Additionally, there are two more paradigms built on top of topics:

- Services: request/response communication. The client node waits (in a blocked state) until the server returns a result.
- Actions: the server provides updates whilst it continues executing, until it finally delivers the result. Non-blocking.

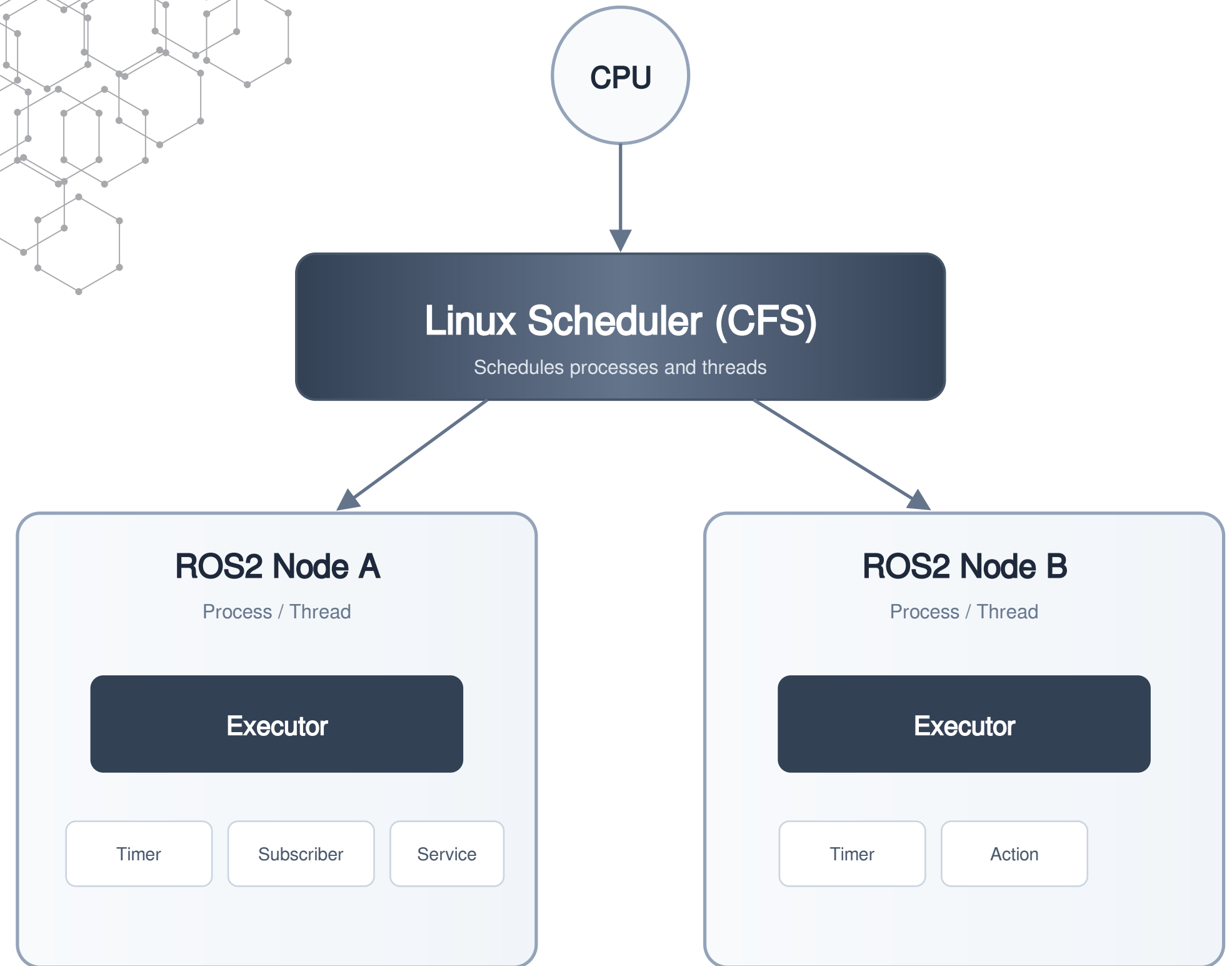
Two-level scheduling

The Linux kernel scheduler views a ROS2 node simply as a standard process or thread. To the CPU, the individual ROS nodes are transparent.

However, a ROS node may have multiple pending internal tasks (callbacks): new message arrival, service request, timer...

That's where the ROS Executors come in.

The executor is essentially a user-space scheduler running on top of the OS scheduler.



Bibliography

The diagrams included in this presentation were created with AI assistance and subsequently reviewed and edited by the author. This material is intended for educational purposes and may be freely reused, modified, and shared with attribution.

ROS Documentation website

Open Robotics. ROS 2 Documentation. Available at:
<https://docs.ros.org/>

ROS robotics projects book

Gandhinathan, R., & Joseph, L. ROS Robotics Projects: Build and Control Robots Powered by the Robot Operating System, Machine Learning, and Virtual Reality. Packt Publishing, 2017.

Course notes

University of Seville. Sistemas Operativos – Course Slides, Laboratory Manual, and Teaching Material (2025–2026).

